Lane-Based Autonomous Obstacle Avoidance Simulator

using x86 Assembly Language



<Computer Architecture(Tue7,Thu56)>

202400555 Shinbi Lee

2026. 06. 05

Table of Contents

I. Introduction

II. Project Goals

III. Tools and Development Environment

- III-1. Programming Language
- III-2. EMU8086
- III-3. DOSBox 0.74-3
- III-4. Video Mode 13h

IV. System Design and Methodology

- IV-1. Main Idea
- IV-2. Overall Program Flow
- IV-3. Game State Variables

V. Implementation Details

- V-1. Graphics Mode and Screen Drawing
- V-2. Main Game Loop
- V-3. Keyboard Control and Game State
- V-4. Obstacle Movement & Random Lane Generation
- V-5. Risk Calculation and Autonomous Lane Decision
- V-6. Collision Detection and HP System
- V-7. HUD and Result Display

VI. Results and Demonstration

VII. Improvements and Extensions

VIII. Key Findings and Benefits

IX. Conclusion

X. References

XI. Appendix: Full Source Code

I. Introduction

The topic of this project is “Lane-Based Autonomous Obstacle Avoidance Simulator using x86 Assembly Language.” The goal is to implement a simple 2D autonomous driving simulator where a vehicle moves between lanes and avoids obstacles.

Unlike many Assembly-based racing games where the user manually controls the vehicle, this project adds ‘lane-based autonomous obstacle avoidance logic’. The program analyzes the road situation, calculates the risk of each lane, selects the safest target lane, and moves the vehicle automatically.

Through this project, I implemented graphics output, a repeated game loop, keyboard input handling, collision detection, random obstacle lane generation, and game state management. I also applied Computer Architecture concepts such as registers, memory, interrupts, conditional branches, loops, and low-level I/O. The program was written and compiled in EMU8086 and executed in DOSBox 0.74-3.

II. Project Goals

The main goal of this project is to implement a working graphics-based simulator using x86 Assembly Language. Instead of creating a simple calculator or text-based program, this project includes screen drawing, object movement, collision checking, and user input handling.

The simulator creates a simple road environment with three lanes. The road is displayed with a gray background and white lane markers, while the player vehicle is shown as a green rectangle and the obstacles are shown as red rectangles.

The project also implements autonomous lane decision logic. The program calculates the risk value of each lane based on obstacle positions, selects the lane with the lowest risk as the target lane, and moves the vehicle toward that lane automatically.

Finally, this project applies low-level concepts learned in the Computer Architecture course. The program uses interrupts, direct video memory access, loop instructions, conditional jumps, register-based calculations, and memory variables to control the simulator.

III. Tools and Development Environment

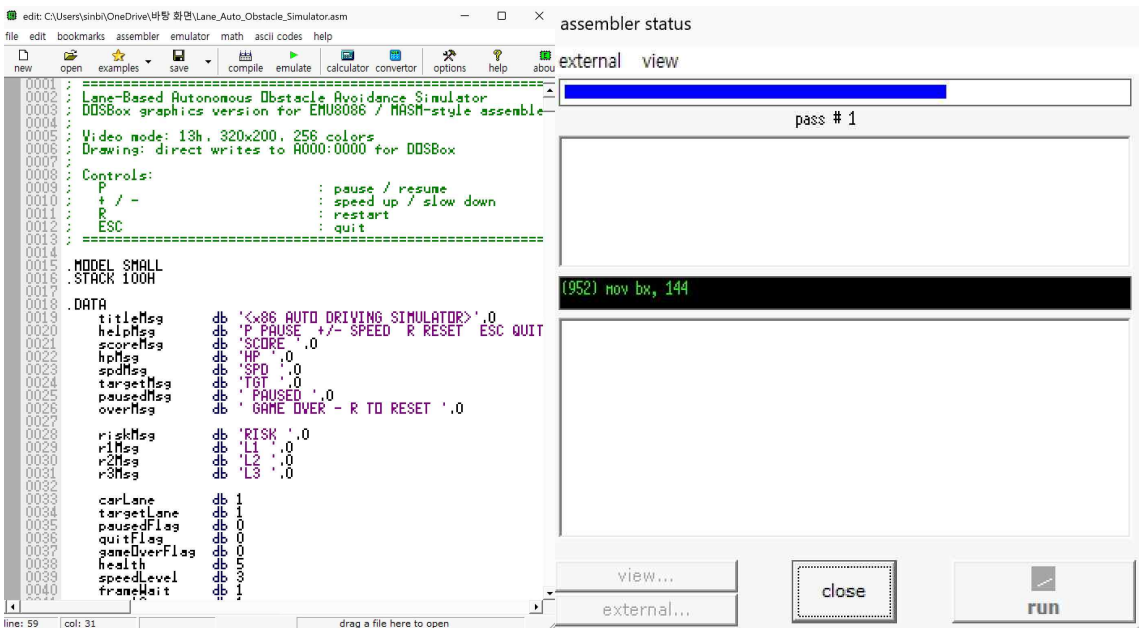
III-1. Programming Language

This project was implemented using x86 Assembly Language. Assembly Language requires direct control of CPU registers, memory, interrupts, and branching. Therefore, it is suitable for applying the main concepts of Computer Architecture.

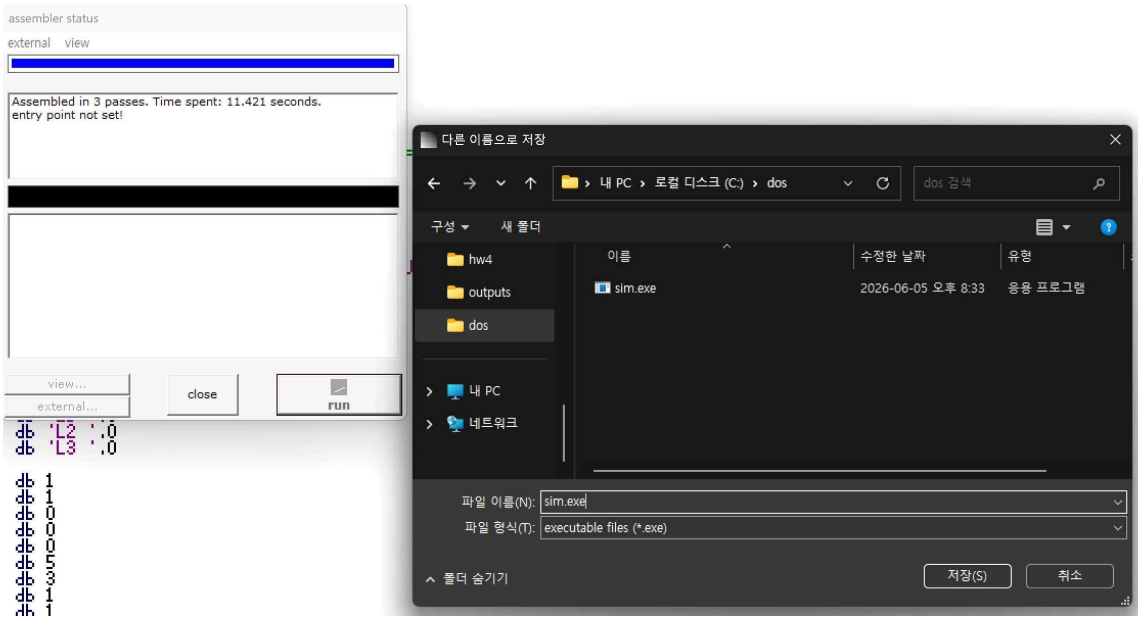
The program was written for a 16-bit DOS environment using '.MODEL SMALL', '.STACK 100H', '.DATA', and '.CODE'. The program state variables are stored in the '.DATA' segment, and the main operations are implemented as procedures in the '.CODE' segment.

III-2. EMU8086

EMU8086 was used to write and compile the Assembly code. The source file 'Lane_Auto_Obstacle_Simulator.asm' was written in EMU8086, and the compile function was used to generate the executable file 'sim.exe'. During compilation, I checked the assembler status window to confirm that the program was compiled successfully.



[Figure 1. Write and compile the ASM code in EMU8086]



[Figure 2. After compiling in EMU8086, save the output file as sim.exe]

III-3. DOSBox 0.74-3

DOSBox 0.74-3 was used as the execution environment. After compiling the program in EMU8086, I mounted the folder containing the executable file as the C drive in DOSBox. Then, I executed the program by entering ‘sim’.

The execution process is as follows.



[Figure 3. Execution of sim.exe in DOSBox after mounting the directory ‘dos’]

III-4. Video Mode 13h

This program uses video mode 13h, which supports a 320×200 resolution with 256 colors. In this mode, graphics can be drawn by writing color values directly to the video memory area ‘A000:0000’.

In this project, the road, lane markers, vehicle, obstacles, pause message, and game-over message are displayed using this graphics mode. The objects are drawn using simple rectangles through the ‘DrawRect’ procedure.

IV. System Design and Methodology

IV-1. Main Idea

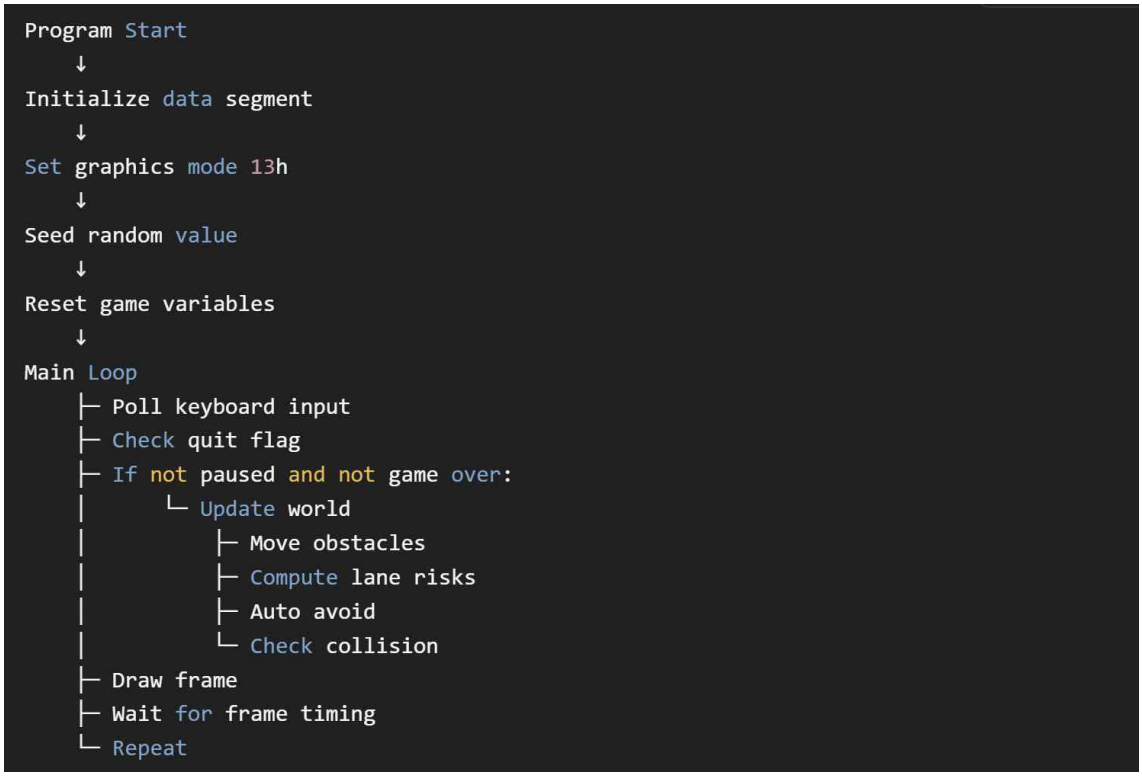
The main idea of this project is to make a vehicle automatically avoid obstacles on a road with three lanes. The vehicle is always located near the bottom of the road, and the obstacles move from the top of the screen to the bottom.

Unlike a normal racing game, the user does not directly control the vehicle. Instead, the program calculates the risk value of each lane and automatically selects the safest target lane. The HUD at the top of the screen shows the score, HP, speed, target lane, and risk values, so the user can see how the program makes its decision.

IV-2. Overall Program Flow

The program is mainly composed of setup, main loop, update, drawing, and timing stages. When the program starts, it initializes the data segment, sets the graphics mode, initializes the random seed, and resets the game variables. Then, it enters the main loop.

The overall flow is as follows.



[Figure 4. Overall Program flow chart]

This structure is a typical game loop implemented in Assembly Language. In a high-level language, this kind of loop can be written using a 'while' or 'for' statement. In Assembly Language, however, repetition and branching must be controlled directly using labels and jump instructions.

IV-3. Game State Variables

The program stores game state variables in the '.DATA' segment. For example, 'carLane' represents the current lane of the vehicle, and 'targetLane' represents the lane selected by the autonomous avoidance logic.

'pausedFlag', 'quitFlag', and 'gameOverFlag' represent the pause, quit, and game-over states. 'health' stores the vehicle's HP, and 'speedLevel' stores the current game speed. The 'obsLane' array stores the lane of each obstacle, and the 'obsY' array stores the y-coordinate of each obstacle. The variables 'risk1', 'risk2', and 'risk3' store the risk values of the three lanes.

These variables are read from memory, processed using registers, and written back to memory. This shows how Assembly programs manage program state without using high-level structures or objects.

V. Implementation Details

V-1. Graphics Mode and Screen Drawing

The graphics mode is set in the 'SetGraphicsMode' procedure.

```
0103 SetGraphicsMode PROC
0104     mov ax, 0013h
0105     int 10h
0106     ret
0107 SetGraphicsMode ENDP
```

In this code, '0013h' is stored in the 'AX' register, and 'int 10h' is called. This changes the screen to video mode 13h.

The program draws the road, vehicle, obstacles, pause box, and game-over box using the 'DrawRect' procedure. In this procedure, the screen position is calculated using the 'BX' and 'DX' registers, and the result is stored in the 'DI' register. Then, 'rep stosb' writes the selected color value repeatedly to video memory.

V-2. Main Game Loop

The main flow of the program is centered around 'MainLoop'. The main loop repeats until the program ends.

```
0074 MainLoop:
0075     call PollKeyboard
0076
0077     cmp byte ptr [quitFlag], 1
0078     je ExitProgram
0079
0080     cmp byte ptr [pausedFlag], 1
0081     je SkipUpdate
0082
0083     cmp byte ptr [gameOverFlag], 1
0084     je SkipUpdate
0085
0086     call UpdateWorld
0087
0088 SkipUpdate:
0089     call DrawFrame
0090     call WaitFrame
0091     jmp MainLoop
```

First, the program checks keyboard input. If the ESC key is pressed, the program exits. If the game is paused or in the game-over state, the game world is not updated. If the game is running normally, the program calls 'UpdateWorld'.

```
0287 UpdateWorld PROC
0288     mov byte ptr [decision], 0
0289     call MoveObstacles
0290     call ComputeRisks
0291     call AutoAvoid
0292     call CheckCollision
0293     ret
0294 UpdateWorld ENDP
```

'UpdateWorld' performs the main game logic: moving obstacles, calculating lane risks, choosing the safest lane, and checking collisions.

V-3. Keyboard Control and Game State

Keyboard input is handled in the 'PollKeyboard' procedure using BIOS interrupt 'int 16h'.

```
0217 KeyLoop:
0218     mov ah, 01h
0219     int 16h
0220     jz KeyDone
0221
0222     mov ah, 00h
0223     int 16h
```

The key value is stored in the 'AL' register. Then, the program compares the key value using 'cmp' and conditional jump instructions.

```
0225     cmp al, 27
0226     je KeyEsc
0227
0228     cmp al, 'p'
0229     je KeyPause
0230     cmp al, 'P'
0231     je KeyPause
0232
0233     cmp al, 'r'
0234     je KeyRestart
0235     cmp al, 'R'
0236     je KeyRestart
0237
0238     cmp al, '+'
0239     je KeyFaster
0240     cmp al, '='
0241     je KeyFaster
0242     cmp al, '-'
0243     je KeySlower
```

The ESC key quits the program, P pauses or resumes the game, R resets the game, and + or - changes the speed. The speed is controlled by changing 'frameWait' and 'pixelStep'. A lower speed level makes the obstacles move more slowly, while a higher speed level makes them move faster.

V-4. Obstacle Movement & Random Lane Generation

The program uses six obstacles. Each obstacle has a lane value stored in ‘obsLane’ and a y-coordinate stored in ‘obsY’. The ‘MoveObstacles’ procedure moves the obstacles downward every frame.

```
0305 MoveObsLoop:
0306     mov al, [pixelStep]
0307     xor ah, ah
0308     add word ptr [obsY+di], ax
0309     cmp word ptr [obsY+di], 180
0310     jbe MoveObsNext
0311
0312     mov word ptr [obsY+di], 30
0313     call NextRandomLane
0314     mov bx, di
0315     shr bx, 1
0316     mov byte ptr [obsLane+bx], al
0317     add word ptr [score], 5
```

When an obstacle passes the bottom of the road, its y-coordinate is reset to the top. Then, ‘NextRandomLane’ assigns a new lane to the obstacle. The score is also increased by 5.

```
0497 NextRandomLane PROC
0498     push bx
0499     push dx
0500
0501     mov ax, [rngSeed]
0502     mov bx, 25173
0503     mul bx
0504     add ax, 13849
0505     mov word ptr [rngSeed], ax
0506
0507     xor dx, dx
0508     mov bx, 3
0509     div bx
0510     mov al, dl
0511
0512     pop dx
0513     pop bx
0514     ret
0515 NextRandomLane ENDP
```

This simple pseudo-random method allows obstacles to appear in different lanes during execution.

V-5. Risk Calculation and Autonomous Lane Decision

The most important feature of this project is the risk-based autonomous lane decision. The ‘ComputeRisks’ procedure calculates the risk values of the three lanes.

```
0330 ComputeRisks PROC
0331     push ax
0332     push bx
0333     push cx
0334     push dx
0335     push si
0336     push di
0337
0338     mov word ptr [risk1], 0
0339     mov word ptr [risk2], 0
0340     mov word ptr [risk3], 0
0341
0342     xor si, si
0343     xor di, di
0344     mov cx, 6
0345
0346 RiskLoop:
0347     mov ax, [obsY+di]
0348     cmp ax, 30
0349     jb RiskNext
0350     cmp ax, 180
0351     ja RiskNext
0352
0353     sub ax, 28
0354     mov bl, [obsLane+si]
0355     cmp bl, 0
0356     je AddRiskOne
0357     cmp bl, 1
0358     je AddRiskTwo
0359     add word ptr [risk3], ax
0360     jmp RiskNext
```

The program checks each obstacle and adds a risk value to the lane where the obstacle is located. A larger y-coordinate means the obstacle is closer to the vehicle, so it creates a higher risk.

After that, the 'AutoAvoid' procedure compares the risk values and selects the lane with the lowest risk.

```
0404 CompareAllRisks:
0405     mov dx, [risk1]
0406     cmp dx, ax
0407     jae CheckRiskTwo
0408     mov ax, dx
0409     mov bl, 0
0410
0411 CheckRiskTwo:
0412     mov dx, [risk2]
0413     cmp dx, ax
0414     jae CheckRiskThree
0415     mov ax, dx
0416     mov bl, 1
0417
0418 CheckRiskThree:
0419     mov dx, [risk3]
0420     cmp dx, ax
0421     jae RiskChoiceDone
0422     mov bl, 2
```

The selected lane is stored in 'targetLane'. Then, the vehicle moves one lane at a time toward the target lane.

```
0424 RiskChoiceDone:
0425     mov byte ptr [targetLane], bl
0426     mov al, [carLane]
0427     cmp al, bl
0428     je AutoHold
0429     jb AutoGoRight
0430
0431 AutoGoLeft:
0432     dec byte ptr [carLane]
0433     mov byte ptr [decision], 1
0434     jmp AutoDone
0435
0436 AutoGoRight:
0437     inc byte ptr [carLane]
0438     mov byte ptr [decision], 2
0439     jmp AutoDone
```

This part makes the project different from a simple racing game. The vehicle does not move by direct user control. Instead, the program calculates the current risk and makes an automatic movement decision.

V-6. Collision Detection and HP System

Collision detection is handled in the 'CheckCollision' procedure. A collision occurs when the obstacle and the vehicle are in the same lane and the obstacle's y-coordinate overlaps with the vehicle area.

```
0462 HitLoop:
0463     mov al, [obsLane+si]
0464     cmp al, byte ptr [carLane]
0465     jne HitNext
0466
0467     mov ax, [obsY+di]
0468     cmp ax, 158
0469     jb HitNext
0470     cmp ax, 188
0471     ja HitNext
```

If a collision occurs, the obstacle is moved back to the top and assigned a new lane. Also, the vehicle's HP is decreased by 1.

```
0477     cmp byte ptr [health], 0
0478     je HitNext
0479     dec byte ptr [health]
0480     cmp byte ptr [health], 0
0481     jne HitNext
0482     mov byte ptr [gameOverFlag], 1
```

When HP becomes 0, 'gameOverFlag' is set to 1, and the game-over message is displayed.

V-7. HUD and Result Display

The program displays a HUD at the top and bottom of the screen. The HUD shows the score, HP, speed, target lane, and risk value of each lane.

SCORE : Current score
HP : Remaining health of the vehicle
SPD : Current speed level
TGT : Target lane selected by the autonomous avoidance logic
RISK : Risk value of each lane

The HUD is important because it shows the internal state of the program. In particular, 'RISK L1', 'RISK L2', 'RISK L3', and 'TGT' help the user understand how the autonomous decision logic works.

The program also displays 'PAUSED' when the P key is pressed and 'GAME OVER - R TO RESET' when HP becomes 0.

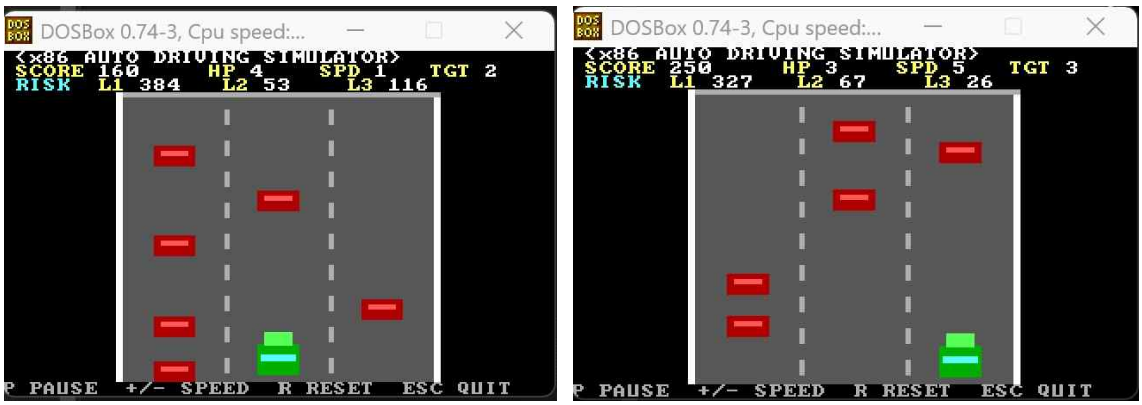
VI. Results and Demonstration

This project was successfully compiled in EMU8086 and executed correctly in DOSBox 0.74-3. The program displayed a road with three lanes, a green vehicle, red obstacles, and a HUD.



[Figure 5. Basic execution screen]

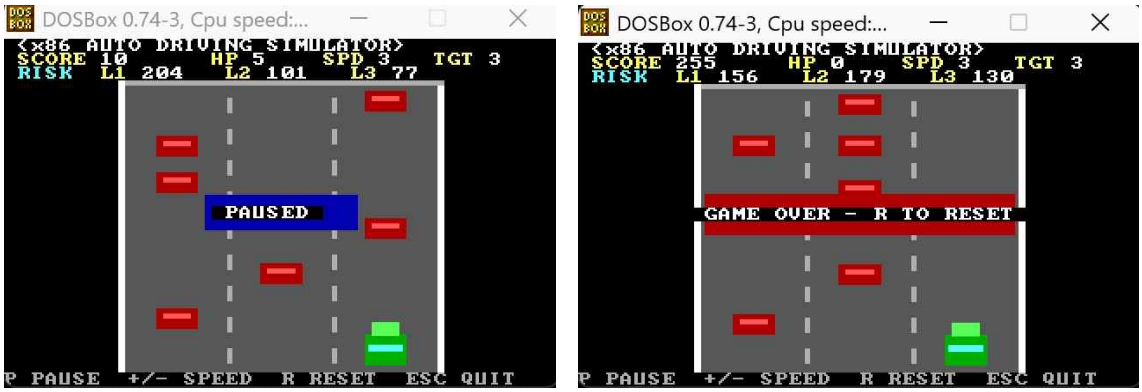
The speed control function worked correctly. At speed level 1, obstacles moved slowly. At speed level 5, obstacles moved faster. This shows that the + and - keys control the game speed as intended.



[Figure 6. Speed level 1 & 5 screen]

The pause function also worked correctly. When the P key was pressed, the 'PAUSED' message appeared in the center of the screen, and the game stopped until the P key was pressed again.

Collision detection and the HP system also worked correctly. When the vehicle collided with obstacles several times and HP became 0, the game-over message appeared on the screen.



[Figure 7. Pause & Game over screen]

The demo video was recorded for about 10 to 15 seconds after executing the program. The video shows the road, vehicle, obstacles, HUD, speed change, pause state, and game-over state. It also shows that the program runs correctly in DOSBox.

VII. Improvements and Extensions Compared to References

This project referred to existing Assembly racing game projects, but it added several extensions instead of simply reproducing them.

A lane-based road system was implemented, where the vehicle and obstacles move within three lanes. In addition, autonomous obstacle avoidance logic was added so that the program calculates lane risks and automatically selects the safest target lane. The risk value is determined not only by the presence of obstacles but also by their distance from the vehicle.

The HUD displays the score, HP, speed, risk values, and target lane, allowing the user to understand the program's decision-making process. Pause, reset, speed control, and game-over functions were also added to make the simulator more complete and interactive.

VIII. Key Findings and Benefits

Through this project, I learned that Assembly Language programming is very different from using a high-level language because the programmer must directly manage registers, memory, interrupts, and program flow.

I learned how to draw graphics by directly accessing video memory. In the 'DrawRect' procedure, screen positions are calculated using registers, and color values are written to video memory. I also learned how BIOS interrupts are used, such as 'int 10h' for graphics and text output, 'int 16h' for keyboard input, and 'int 1Ah' for random seed initialization.

In addition, I understood how a game loop is implemented using labels, procedure calls, and jump instructions. The program repeatedly checks input, updates the world, draws the screen, and waits for the next frame. Program states such as vehicle lane, obstacle positions, HP, speed, and risk values are stored in memory, while calculations are performed using registers.

Overall, this project helped me connect Computer Architecture concepts with a working program and showed how low-level instructions can be used to build an interactive graphics application.

IX. Conclusion

In this project, I implemented a lane-based autonomous obstacle avoidance simulator using x86 Assembly Language. The program was written and compiled in EMU8086 and executed successfully in DOSBox 0.74-3.

The implemented program displays a three-lane road, a vehicle, obstacles, and a HUD in graphics mode. The obstacles move downward, and the vehicle automatically selects the safest lane by comparing the risk values of the three lanes. The program also includes collision detection, HP decrease, game over, pause, reset, speed control, and HUD display functions.

This project was meaningful because it was not only a reproduction of a simple racing game. It added autonomous driving-inspired decision logic and showed how Assembly Language can be used to implement an interactive simulator. Through this project, I gained a better understanding of how CPU instructions, memory, registers, interrupts, and branches are used in program execution.

X. References

[1] Mahmoud Khaled Nasr, “Car-Racing-Game-x8086,” GitHub.
https://github.com/Mahmoud-Khaled-Nasr/Car-Racing-Game-x8086

[2] Youssef Tawakal, “driving-car-asm,” GitHub.
https://github.com/yousseftawakal/driving-car-asm

[3] NASM Official Documentation.
https://www.nasm.us/docs.php

[4] Visual Studio Code Official Website.
https://code.visualstudio.com/

[5] Yousef Kotp, “8086-Assembly-Projects,” GitHub.
https://github.com/yousefkotp/8086-Assembly-Projects

XI. Appendix: Full Source Code

The complete source code of this project consists of about 1000 lines. In the main body of the report, the most important code sections were explained, including the main loop, graphics setup, keyboard input, obstacle movement, risk calculation, autonomous lane decision, collision detection, and HUD display. The full code is included in the Appendix so that the complete implementation can be checked.

```
; ================================================================
; Lane-Based Autonomous Obstacle Avoidance Simulator
; DOSBox graphics version for EMU8086 / MASM-style assemblers
;
; Video mode: 13h, 320x200, 256 colors
; Drawing: direct writes to A000:0000 for DOSBox
;
; Controls:
;   P                       : pause / resume
;   + / -                   : speed up / slow down
;   R                       : restart
;   ESC                     : quit
; ================================================================

.MODEL SMALL
.STACK 100H

.DATA
    titleMsg      db '<x86 AUTO DRIVING SIMULATOR>',0
    helpMsg       db 'P PAUSE  +/- SPEED  R RESET  ESC QUIT',0
    scoreMsg      db 'SCORE ',0
    hpMsg         db 'HP ',0
    spdMsg        db 'SPD ',0
    targetMsg     db 'TGT ',0
    pausedMsg     db ' PAUSED ',0
    overMsg       db ' GAME OVER - R TO RESET ',0

    riskMsg       db 'RISK ',0
    r1Msg         db 'L1 ',0
    r2Msg         db 'L2 ',0
    r3Msg         db 'L3 ',0

    carLane       db 1
    targetLane    db 1
    pausedFlag    db 0
    quitFlag      db 0
    gameOverFlag  db 0
    health        db 5
    speedLevel    db 3
    frameWait     db 1
    pixelStep     db 1
    decision      db 0
    redrawFlag    db 1

    score         dw 0
    rngSeed       dw 0A317h

    laneX         dw 100, 160, 220
    obsLane       db 0, 2, 1, 0, 2, 1
    obsY          dw 32, 58, 84, 110, 136, 162
    prevCarLane   db 1
    prevObsLane   db 0, 2, 1, 0, 2, 1
    prevObsY      dw 32, 58, 84, 110, 136, 162

    risk1         dw 0
    risk2         dw 0
    risk3         dw 0

    rectW         dw 0
    rectColor     db 0
    numColor      db 0
    numBuffer     db 6 dup(0)

.CODE
MAIN PROC
    mov ax, @DATA
    mov ds, ax

    call SetGraphicsMode
    call SeedRandom
    call ResetGame

MainLoop:
    call PollKeyboard

    cmp byte ptr [quitFlag], 1
    je ExitProgram

    cmp byte ptr [pausedFlag], 1
    je SkipUpdate

    cmp byte ptr [gameOverFlag], 1
    je SkipUpdate

    call UpdateWorld

SkipUpdate:
    call DrawFrame
    call WaitFrame
    jmp MainLoop

ExitProgram:
    mov ax, 0003h
    int 10h
    mov ah, 4Ch
    int 21h
MAIN ENDP

; ================================================================
; Setup
; ================================================================
```

```
SetGraphicsMode PROC
    mov ax, 0013h
    int 10h
    ret
SetGraphicsMode ENDP

SeedRandom PROC
    push ax
    push cx
    push dx

    mov ah, 00h
    int 1Ah
    mov word ptr [rngSeed], dx

    pop dx
    pop cx
    pop ax
    ret
SeedRandom ENDP

ResetGame PROC
    mov byte ptr [carLane], 1
    mov byte ptr [targetLane], 1
    mov byte ptr [pausedFlag], 0
    mov byte ptr [quitFlag], 0
    mov byte ptr [gameOverFlag], 0
    mov byte ptr [health], 5
    mov byte ptr [speedLevel], 3
    mov byte ptr [frameWait], 1
    mov byte ptr [pixelStep], 1
    mov byte ptr [decision], 0
    mov byte ptr [redrawFlag], 1

    mov word ptr [score], 0

    mov byte ptr [obsLane], 0
    mov byte ptr [obsLane+1], 2
    mov byte ptr [obsLane+2], 1
    mov byte ptr [obsLane+3], 0
    mov byte ptr [obsLane+4], 2
    mov byte ptr [obsLane+5], 1

    mov word ptr [obsY], 32
    mov word ptr [obsY+2], 58
    mov word ptr [obsY+4], 84
    mov word ptr [obsY+6], 110
    mov word ptr [obsY+8], 136
    mov word ptr [obsY+10], 162

    mov byte ptr [prevCarLane], 1
    mov byte ptr [prevObsLane], 0
    mov byte ptr [prevObsLane+1], 2
    mov byte ptr [prevObsLane+2], 1
    mov byte ptr [prevObsLane+3], 0
    mov byte ptr [prevObsLane+4], 2
    mov byte ptr [prevObsLane+5], 1

    mov word ptr [prevObsY], 32
    mov word ptr [prevObsY+2], 58
    mov word ptr [prevObsY+4], 84
    mov word ptr [prevObsY+6], 110
    mov word ptr [prevObsY+8], 136
    mov word ptr [prevObsY+10], 162
    ret
ResetGame ENDP

ApplySpeed PROC
    push ax

    mov al, [speedLevel]
    cmp al, 1
    je SpeedOne
    cmp al, 2
    je SpeedTwo
    cmp al, 3
    je SpeedThree
    cmp al, 4
    je SpeedFour

    mov byte ptr [frameWait], 1
    mov byte ptr [pixelStep], 3
    jmp SpeedDone

SpeedOne:
    mov byte ptr [frameWait], 4
    mov byte ptr [pixelStep], 1
    jmp SpeedDone

SpeedTwo:
    mov byte ptr [frameWait], 3
    mov byte ptr [pixelStep], 1
    jmp SpeedDone

SpeedThree:
    mov byte ptr [frameWait], 1
    mov byte ptr [pixelStep], 1
    jmp SpeedDone

SpeedFour:
    mov byte ptr [frameWait], 1
    mov byte ptr [pixelStep], 2

SpeedDone:
    pop ax
    ret
ApplySpeed ENDP

; ================================================================
; Keyboard
; ================================================================
PollKeyboard PROC
    push ax

KeyLoop:
    mov ah, 01h
    int 16h
    jz KeyDone

    mov ah, 00h
    int 16h

    cmp al, 27
    je KeyEsc

    cmp al, 'p'
    je KeyPause
    cmp al, 'P'
    je KeyPause

    cmp al, 'r'
    je KeyRestart
    cmp al, 'R'
    je KeyRestart
```

```
    cmp al, '+'
    je KeyFaster
    cmp al, '='
    je KeyFaster
    cmp al, '-'
    je KeySlower

    jmp KeyLoop

KeyEsc:
    mov byte ptr [quitFlag], 1
    jmp KeyDone

KeyPause:
    cmp byte ptr [pausedFlag], 0
    je PauseOn
    mov byte ptr [pausedFlag], 0
    mov byte ptr [redrawFlag], 1
    jmp KeyLoop
PauseOn:
    mov byte ptr [pausedFlag], 1
    jmp KeyLoop

KeyRestart:
    call ResetGame
    jmp KeyLoop

KeyFaster:
    cmp byte ptr [speedLevel], 5
    jae KeyLoop
    inc byte ptr [speedLevel]
    call ApplySpeed
    jmp KeyLoop

KeySlower:
    cmp byte ptr [speedLevel], 1
    jbe KeyLoop
    dec byte ptr [speedLevel]
    call ApplySpeed
    jmp KeyLoop

KeyDone:
    pop ax
    ret
PollKeyboard ENDP

; ============================================================
; Game logic
; ============================================================
UpdateWorld PROC
    mov byte ptr [decision], 0
    call MoveObstacles
    call ComputeRisks
    call AutoAvoid
    call CheckCollision
    ret
UpdateWorld ENDP

MoveObstacles PROC
    push ax
    push bx
    push cx
    push di

    xor di, di
    mov cx, 6

MoveObsLoop:
    mov al, [pixelStep]
    xor ah, ah
    add word ptr [obsY+di], ax
    cmp word ptr [obsY+di], 180
    jbe MoveObsNext

    mov word ptr [obsY+di], 30
    call NextRandomLane
    mov bx, di
    shr bx, 1
    mov byte ptr [obsLane+bx], al
    add word ptr [score], 5

MoveObsNext:
    add di, 2
    loop MoveObsLoop

    pop di
    pop cx
    pop bx
    pop ax
    ret
MoveObstacles ENDP

ComputeRisks PROC
    push ax
    push bx
    push cx
    push dx
    push si
    push di

    mov word ptr [risk1], 0
    mov word ptr [risk2], 0
    mov word ptr [risk3], 0

    xor si, si
    xor di, di
    mov cx, 6

RiskLoop:
    mov ax, [obsY+di]
    cmp ax, 30
    jb RiskNext
    cmp ax, 180
    ja RiskNext

    sub ax, 28
    mov bl, [obsLane+si]
    cmp bl, 0
    je AddRiskOne
    cmp bl, 1
    je AddRiskTwo
    add word ptr [risk3], ax
    jmp RiskNext

AddRiskOne:
    add word ptr [risk1], ax
    jmp RiskNext

AddRiskTwo:
    add word ptr [risk2], ax

RiskNext:
    inc si
    add di, 2
    loop RiskLoop
```

```
    pop di
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
ComputeRisks ENDP

AutoAvoid PROC
    push ax
    push bx
    push dx

    mov al, [carLane]
    mov bl, al
    cmp al, 0
    je BaseRiskOne
    cmp al, 1
    je BaseRiskTwo
    mov ax, [risk3]
    jmp CompareAllRisks

BaseRiskOne:
    mov ax, [risk1]
    jmp CompareAllRisks

BaseRiskTwo:
    mov ax, [risk2]

CompareAllRisks:
    mov dx, [risk1]
    cmp dx, ax
    jae CheckRiskTwo
    mov ax, dx
    mov bl, 0

CheckRiskTwo:
    mov dx, [risk2]
    cmp dx, ax
    jae CheckRiskThree
    mov ax, dx
    mov bl, 1

CheckRiskThree:
    mov dx, [risk3]
    cmp dx, ax
    jae RiskChoiceDone
    mov bl, 2

RiskChoiceDone:
    mov byte ptr [targetLane], bl
    mov al, [carLane]
    cmp al, bl
    je AutoHold
    jb AutoGoRight

AutoGoLeft:
    dec byte ptr [carLane]
    mov byte ptr [decision], 1
    jmp AutoDone

AutoGoRight:
    inc byte ptr [carLane]
    mov byte ptr [decision], 2
    jmp AutoDone

AutoHold:
    mov byte ptr [decision], 0

AutoDone:
    pop dx
    pop bx
    pop ax
    ret
AutoAvoid ENDP

CheckCollision PROC
    push ax
    push bx
    push cx
    push si
    push di

    xor si, si
    xor di, di
    mov cx, 6

HitLoop:
    mov al, [obsLane+si]
    cmp al, byte ptr [carLane]
    jne HitNext

    mov ax, [obsY+di]
    cmp ax, 158
    jb HitNext
    cmp ax, 188
    ja HitNext

    mov word ptr [obsY+di], 30
    call NextRandomLane
    mov byte ptr [obsLane+si], al

    cmp byte ptr [health], 0
    je HitNext
    dec byte ptr [health]
    cmp byte ptr [health], 0
    jne HitNext
    mov byte ptr [gameOverFlag], 1

HitNext:
    inc si
    add di, 2
    loop HitLoop

    pop di
    pop si
    pop cx
    pop bx
    pop ax
    ret
CheckCollision ENDP

NextRandomLane PROC
    push bx
    push dx
```

```
    mov ax, [rngSeed]
    mov bx, 25173
    mul bx
    add ax, 13849
    mov word ptr [rngSeed], ax

    xor dx, dx
    mov bx, 3
    div bx
    mov al, dl

    pop dx
    pop bx
    ret
NextRandomLane ENDP

; ============================================================
; Drawing
; ============================================================
DrawFrame PROC
    mov ax, 0A000h
    mov es, ax

    cmp byte ptr [redrawFlag], 1
    je DrawFullFrame

    call ErasePreviousSprites
    jmp DrawMovingParts

DrawFullFrame:
    call ClearGfx
    call DrawRoad
    call DrawLaneMarkers
    call DrawStaticHud
    mov byte ptr [redrawFlag], 0

DrawMovingParts:
    call DrawObstacles
    call DrawPlayer
    call DrawDynamicHud

    cmp byte ptr [pausedFlag], 1
    jne NotPausedDraw
    call DrawPauseOverlay
NotPausedDraw:

    cmp byte ptr [gameOverFlag], 1
    jne NotOverDraw
    call DrawGameOverOverlay
NotOverDraw:
    call SaveSpritePositions
    ret
DrawFrame ENDP

ClearGfx PROC
    push ax
    push cx
    push di
    push es

    mov ax, 0A000h
    mov es, ax
    xor di, di
    xor ax, ax
    mov cx, 32000
    cld
    rep stosw

    pop es
    pop di
    pop cx
    pop ax
    ret
ClearGfx ENDP

DrawRoad PROC
    push ax
    push bx
    push cx
    push dx
    push si

    mov bx, 70
    mov dx, 24
    mov cx, 180
    mov si, 168
    mov al, 8
    call DrawRect

    mov bx, 66
    mov dx, 24
    mov cx, 4
    mov si, 168
    mov al, 15
    call DrawRect

    mov bx, 250
    mov dx, 24
    mov cx, 4
    mov si, 168
    mov al, 15
    call DrawRect

    mov bx, 70
    mov dx, 24
    mov cx, 184
    mov si, 3
    mov al, 7
    call DrawRect

    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
DrawRoad ENDP

DrawLaneMarkers PROC
    push ax
    push bx
    push cx
    push dx
    push si
    push di

    mov di, 34

LaneDashLoop:
    cmp di, 184
    ja LaneDashDone
```

```
    mov bx, 129
    mov dx, di
    mov cx, 3
    mov si, 9
    mov al, 7
    call DrawRect

    mov bx, 189
    mov dx, di
    mov cx, 3
    mov si, 9
    mov al, 7
    call DrawRect

    add di, 18
    jmp LaneDashLoop

LaneDashDone:
    pop di
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
DrawLaneMarkers ENDP

DrawObstacles PROC
    push ax
    push bx
    push cx
    push dx
    push si
    push di

    xor si, si
    xor di, di
    mov cx, 6

DrawObsLoop:
    push cx
    push si

    mov al, [obsLane+si]
    call LaneCenterToBX
    sub bx, 12
    mov dx, [obsY+di]
    mov cx, 24
    mov si, 12
    mov al, 4
    call DrawRect

    add bx, 4
    add dx, 3
    mov cx, 16
    mov si, 3
    mov al, 12
    call DrawRect

    pop si
    pop cx
    inc si
    add di, 2
    loop DrawObsLoop

    pop di
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
DrawObstacles ENDP

DrawPlayer PROC
    push ax
    push bx
    push cx
    push dx
    push si

    mov al, [carLane]
    call LaneCenterToBX
    sub bx, 12

    mov dx, 170
    mov cx, 24
    mov si, 18
    mov al, 2
    call DrawRect

    add bx, 4
    sub dx, 7
    mov cx, 16
    mov si, 8
    mov al, 10
    call DrawRect

    sub bx, 2
    add dx, 13
    mov cx, 20
    mov si, 4
    mov al, 11
    call DrawRect

    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
DrawPlayer ENDP

ErasePreviousSprites PROC
    push ax
    push bx
    push cx
    push dx
    push si
    push di

    xor si, si
    xor di, di
    mov cx, 6

```

```
EraseObsLoop:
    push cx
    push si

    mov al, [prevObsLane+si]
    call LaneCenterToBX
    sub bx, 12
    mov dx, [prevObsY+di]
    mov cx, 24
    mov si, 12
    mov al, 8
    call DrawRect

    pop si
    pop cx
    inc si
    add di, 2
    loop EraseObsLoop

    mov al, [prevCarLane]
    call LaneCenterToBX
    sub bx, 12
    mov dx, 163
    mov cx, 24
    mov si, 25
    mov al, 8
    call DrawRect

    pop di
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
ErasePreviousSprites ENDP

SaveSpritePositions PROC
    push ax
    push cx
    push si
    push di

    mov al, [carLane]
    mov byte ptr [prevCarLane], al

    xor si, si
    xor di, di
    mov cx, 6

SaveSpriteLoop:
    mov al, [obsLane+si]
    mov byte ptr [prevObsLane+si], al
    mov ax, [obsY+di]
    mov word ptr [prevObsY+di], ax
    inc si
    add di, 2
    loop SaveSpriteLoop

    pop di
    pop si
    pop cx
    pop ax
    ret
SaveSpritePositions ENDP

DrawStaticHud PROC
    push ax
    push bx
    push dx
    push si

    mov dh, 0
    mov dl, 1
    mov bl, 15
    lea si, titleMsg
    call PrintString

    mov dh, 1
    mov dl, 1
    mov bl, 14
    lea si, scoreMsg
    call PrintString
    mov ax, [score]
    mov dh, 1
    mov dl, 7
    mov bl, 15
    call PrintNumber

    mov dh, 1
    mov dl, 15
    mov bl, 14
    lea si, hpMsg
    call PrintString
    mov al, [health]
    xor ah, ah
    mov dh, 1
    mov dl, 18
    mov bl, 15
    call PrintNumber

    mov dh, 1
    mov dl, 23
    mov bl, 14
    lea si, spdMsg
    call PrintString
    mov al, [speedLevel]
    xor ah, ah
    mov dh, 1
    mov dl, 27
    mov bl, 15
    call PrintNumber

    mov dh, 2
    mov dl, 1
    mov bl, 11
    lea si, riskMsg
    call PrintString

    mov dh, 2
    mov dl, 7
    mov bl, 14
    lea si, r1Msg
    call PrintString
    mov ax, [risk1]
    mov dh, 2
    mov dl, 10
    mov bl, 15
    call PrintNumber
```

```
    mov dh, 2
    mov dl, 16
    mov bl, 14
    lea si, r2Msg
    call PrintString
    mov ax, [risk2]
    mov dh, 2
    mov dl, 19
    mov bl, 15
    call PrintNumber

    mov dh, 2
    mov dl, 25
    mov bl, 14
    lea si, r3Msg
    call PrintString
    mov ax, [risk3]
    mov dh, 2
    mov dl, 28
    mov bl, 15
    call PrintNumber

    mov dh, 1
    mov dl, 31
    mov bl, 14
    lea si, targetMsg
    call PrintString
    mov al, [targetLane]
    inc al
    xor ah, ah
    mov dh, 1
    mov dl, 35
    mov bl, 15
    call PrintNumber

    mov dh, 24
    mov dl, 0
    mov bl, 7
    lea si, helpMsg
    call PrintString

    pop si
    pop dx
    pop bx
    pop ax
    ret
DrawStaticHud ENDP

DrawDynamicHud PROC
    push ax
    push bx
    push cx
    push dx
    push si

    mov bx, 56
    mov dx, 8
    mov cx, 48
    mov si, 8
    mov al, 0
    call DrawRect

    mov bx, 144
    mov dx, 8
    mov cx, 16
    mov si, 8
    mov al, 0
    call DrawRect

    mov bx, 216
    mov dx, 8
    mov cx, 16
    mov si, 8
    mov al, 0
    call DrawRect

    mov bx, 280
    mov dx, 8
    mov cx, 8
    mov si, 8
    mov al, 0
    call DrawRect

    mov bx, 80
    mov dx, 16
    mov cx, 32
    mov si, 8
    mov al, 0
    call DrawRect

    mov bx, 152
    mov dx, 16
    mov cx, 32
    mov si, 8
    mov al, 0
    call DrawRect

    mov bx, 224
    mov dx, 16
    mov cx, 32
    mov si, 8
    mov al, 0
    call DrawRect

    mov ax, [score]
    mov dh, 1
    mov dl, 7
    mov bl, 15
    call PrintNumber

    mov al, [health]
    xor ah, ah
    mov dh, 1
    mov dl, 18
    mov bl, 15
    call PrintNumber

    mov al, [speedLevel]
    xor ah, ah
    mov dh, 1
    mov dl, 27
    mov bl, 15
    call PrintNumber

    mov al, [targetLane]
    inc al
    xor ah, ah
    mov dh, 1
    mov dl, 35
    mov bl, 15
    call PrintNumber
```

```
    mov ax, [risk1]
    mov dh, 2
    mov dl, 10
    mov bl, 15
    call PrintNumber

    mov ax, [risk2]
    mov dh, 2
    mov dl, 19
    mov bl, 15
    call PrintNumber

    mov ax, [risk3]
    mov dh, 2
    mov dl, 28
    mov bl, 15
    call PrintNumber

    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
DrawDynamicHud ENDP

DrawPauseOverlay PROC
    push ax
    push bx
    push cx
    push dx
    push si

    mov bx, 116
    mov dx, 90
    mov cx, 88
    mov si, 20
    mov al, 1
    call DrawRect

    mov dh, 12
    mov dl, 15
    mov bl, 15
    lea si, pausedMsg
    call PrintString

    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
DrawPauseOverlay ENDP

DrawGameOverOverlay PROC
    push ax
    push bx
    push cx
    push dx
    push si

    mov bx, 72
    mov dx, 88
    mov cx, 176
    mov si, 24
    mov al, 4
    call DrawRect

    mov dh, 12
    mov dl, 8
    mov bl, 15
    lea si, overMsg
    call PrintString

    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
DrawGameOverOverlay ENDP

; Input: BX=x, DX=y, CX=width, SI=height, AL=color
DrawRect PROC
    push ax
    push bx
    push cx
    push dx
    push si
    push di
    push es

    mov byte ptr [rectColor], al
    mov word ptr [rectW], cx

    mov ax, 0A000h
    mov es, ax

    mov ax, dx
    mov di, ax
    shl di, 8
    mov ax, dx
    shl ax, 6
    add di, ax
    add di, bx

DrawRectRow:
    cmp si, 0
    je DrawRectDone
    mov cx, word ptr [rectW]
    mov al, byte ptr [rectColor]
    cld
    rep stosb
    add di, 320
    sub di, word ptr [rectW]
    dec si
    jmp DrawRectRow

DrawRectDone:
    pop es
    pop di
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
DrawRect ENDP

```

```
; Input: AL=lane index, Output: BX=center x
LaneCenterToBX PROC
    push ax
    xor ah, ah
    shl ax, 1
    mov bx, ax
    mov bx, word ptr [laneX+bx]
    pop ax
    ret
LaneCenterToBX ENDP

; ================================================================
; Text output in mode 13h
; ================================================================
; Input: DH=row, DL=col, SI=zero-terminated string, BL=color
PrintString PROC
    push ax
    push bx
    push cx
    push dx
    push si

    mov ah, 02h
    mov bh, 0
    int 10h

TextLoop:
    lodsb
    cmp al, 0
    je TextDone
    mov ah, 0Eh
    mov bh, 0
    int 10h
    jmp TextLoop

TextDone:
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
PrintString ENDP

; Input: AX=number, DH=row, DL=col, BL=color
PrintNumber PROC
    push ax
    push bx
    push cx
    push dx
    push si
    push di

    mov byte ptr [numColor], bl
    mov di, dx
    lea si, numBuffer+5
    mov byte ptr [si], 0
    mov bx, 10

    cmp ax, 0
    jne NumberLoop

    dec si
    mov byte ptr [si], '0'
    jmp NumberPrint

NumberLoop:
    xor dx, dx
    div bx
    add dl, '0'
    dec si
    mov byte ptr [si], dl
    cmp ax, 0
    jne NumberLoop

NumberPrint:
    mov dx, di
    mov bl, byte ptr [numColor]
    call PrintString

    pop di
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
PrintNumber ENDP

; ================================================================
; Frame timing
; ================================================================
WaitFrame PROC
    push ax
    push cx
    push dx

    mov cl, [frameWait]
    xor ch, ch

WaitFrameLoop:
    call WaitRetrace
    call PollKeyboard
    cmp byte ptr [quitFlag], 1
    je WaitFrameDone
    loop WaitFrameLoop

WaitFrameDone:
    pop dx
    pop cx
    pop ax
    ret
WaitFrame ENDP

WaitRetrace PROC
    push ax
    push dx

    mov dx, 03DAh

WaitRetraceEnd:
    in al, dx
    test al, 08h
    jnz WaitRetraceEnd

WaitRetraceStart:
    in al, dx
    test al, 08h
    jz WaitRetraceStart

    pop dx
    pop ax
    ret
WaitRetrace ENDP

END MAIN
```